

Лабораторная работа № 8

Тема: Валидация входных данных и обработка ошибок

Цель работы: Добавить в **Internet Shop API** проверку данных для операций создания и изменения товара, реализовать передачу ошибок через `next(error)` и создать централизованный `middleware` обработки ошибок.

Практическое задание

Продолжите проект предыдущей лабораторной работы.

После выполнения структура должна включать:

`express-shop/`

```
|— controllers/
|   |— productController.js
|— errors/
|   |— AppError.js
|— middleware/
|   |— errorHandler.js
|   |— notFound.js
|   |— validateCreateProduct.js
|   |— validateUpdateProduct.js
|— routes/
|   |— productRoutes.js
|— services/
|   |— productService.js
|— validators/
|   |— productValidator.js
|— app.js
|— package.json
```

1. Создайте функцию проверки товара

Создайте:

`validators/productValidator.js`

Добавьте:

```
export function validateProductData(data) {
  const errors = [];
```

```
  if (
    typeof data.name !== "string" ||
    !data.name.trim()
  ) {
    errors.push("name is required");
  }
```

```
  if (
    typeof data.price !== "number" ||
    data.price <= 0
```

```

    ) {
      errors.push(
        "price must be greater than 0"
      );
    }

    if (
      typeof data.category !== "string" ||
      !data.category.trim()
    ) {
      errors.push("category is required");
    }

    if (typeof data.inStock !== "boolean") {
      errors.push(
        "inStock must be boolean"
      );
    }

    return errors;
  }

```

2. Создайте middleware для POST

Создайте:

middleware/validateCreateProduct.js

```

import {
  validateProductData
} from "../validators/productValidator.js";

```

```

export function validateCreateProduct(
  req,
  res,
  next
) {
  const errors =
    validateProductData(req.body);

  if (errors.length > 0) {
    return res.status(400).json({
      message: "Validation failed",
      errors
    });
  }

  next();
}

```

3. Подключите validation middleware

В productRoutes.js:

```
router.post(
  "/",
  validateCreateProduct,
  createProduct
);
```

Последовательность должна быть:

POST

↓

validateCreateProduct

↓

createProduct

4. Проверьте корректный POST

Отправьте:

POST /api/products

```
{
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true
}
```

Ожидается:

201 Created

5. Проверьте отсутствие name

Отправьте:

```
{
  "price": 65000,
  "category": "Components",
  "inStock": true
}
```

Ожидается:

400 Bad Request

Например:

```
{
  "message": "Validation failed",
  "errors": [
    "name is required"
  ]
}
```

6. Проверьте отрицательную цену

```
{
  "name": "SSD 2TB",
  "price": -100,
```

```
"category": "Components",  
"inStock": true  
}
```

Запрос должен быть отклонён.

7. Проверьте несколько ошибок одновременно

```
{  
  "name": "",  
  "price": -100,  
  "category": 123,  
  "inStock": "yes"  
}
```

API должен вернуть несколько сообщений в массиве:

errors

Товар не должен добавляться в products.

8. Создайте валидацию PATCH

Создайте:

middleware/validateUpdateProduct.js

```
export function validateUpdateProduct(  
  req,  
  res,  
  next  
) {  
  const {  
    name,  
    price,  
    category,  
    inStock  
  } = req.body;
```

```
  const errors = [];
```

```
  if (  
    name !== undefined &&  
    (  
      typeof name !== "string" ||  
      !name.trim()  
    )  
  ) {  
    errors.push(  
      "name must be non-empty string"  
    );  
  }  
}
```

```
  if (  
    price !== undefined &&
```

```

    (
      typeof price !== "number" ||
      price <= 0
    )
  ) {
    errors.push(
      "price must be greater than 0"
    );
  }

  if (
    category !== undefined &&
    (
      typeof category !== "string" ||
      !category.trim()
    )
  ) {
    errors.push(
      "category must be non-empty string"
    );
  }

  if (
    inStock !== undefined &&
    typeof inStock !== "boolean"
  ) {
    errors.push(
      "inStock must be boolean"
    );
  }

  if (errors.length > 0) {
    return res.status(400).json({
      message: "Validation failed",
      errors
    });
  }

  next();
}

```

9. Подключите PATCH validation

```

router.patch(
  "/:id",
  validateUpdateProduct,
  updateProduct
);

```

10. Проверьте частичное обновление

Допустимый запрос:

```
{  
  "price": 48000  
}
```

Не нужно передавать name, category и inStock.

11. Проверьте ошибочный PATCH

```
{  
  "price": -9000  
}
```

Ожидается:

400 Bad Request

Текущая цена товара не должна измениться.

12. Создайте AppError

Создайте:

errors/AppError.js

```
export class AppError extends Error {  
  constructor(  
    message,  
    statusCode = 500  
  ) {  
    super(message);  
  
    this.statusCode = statusCode;  
  }  
}
```

13. Измените getProductById

Импортируйте AppError и используйте next:

```
export function getProductById(  
  req,  
  res,  
  next  
) {  
  const id =  
    Number(req.params.id);  
  
  const product =  
    findProductById(id);  
  
  if (!product) {  
    return next(  
      new AppError(  
        "Product not found",
```

```

        404
      )
    );
  }

  res.json(product);
}

```

14. Аналогично измените `updateProduct`

При отсутствии товара:

```

return next(
  new AppError(
    "Product not found",
    404
  )
);

```

15. Аналогично измените `deleteProduct`

Отсутствующий товар также должен передавать ошибку через:

`next(error)`

а не самостоятельно формировать 404.

16. Создайте `errorHandler`

Создайте:

`middleware/errorHandler.js`

```

export function errorHandler(
  err,
  req,
  res,
  next
) {
  const statusCode =
    err.statusCode || 500;

  console.error(err);

  res
    .status(statusCode)
    .json({
      message:
        statusCode === 500
          ? "Internal server error"
          : err.message
    });
}

```

Error-handling middleware Express отличается от обычного middleware сигнатурой из четырёх параметров (err, req, res, next) и обычно регистрируется после остальных маршрутов и middleware.

17. Создайте обработчик неизвестного маршрута

Создайте:

middleware/notFound.js

```
import {
  AppError
} from "../errors/AppError.js";

export function notFound(
  req,
  res,
  next
) {
  next(
    new AppError(
      `Route ${req.method} ${req.originalUrl} not found`,
      404
    )
  );
}
```

18. Подключите middleware в app.js

Порядок должен быть таким:

```
app.use(express.json());
```

```
app.use(
  "/api/products",
  productRoutes
);
```

```
app.use(notFound);
```

```
app.use(errorHandler);
```

Error handler должен находиться в конце цепочки; next(error) пропускает обычные обработчики и передаёт управление обработчикам ошибок.

19. Проверьте неизвестный ресурс

GET /api/products/999

Ожидается:

404 Not Found

```
{
  "message": "Product not found"
}
```

20. Проверьте неизвестный маршрут

GET /api/unknown

Ожидается:

404 Not Found

21. Проверьте внутреннюю ошибку

Временно создайте тестовый маршрут:

```
router.get(
  "/test/error",
  (req, res, next) => {
    try {
      throw new Error(
        "Test server error"
      );
    } catch (error) {
      next(error);
    }
  }
);
```

Ожидается:

500 Internal Server Error

Клиент должен получить:

```
{
  "message": "Internal server error"
}
```

Статусы 400–499 относятся к ошибкам клиента, а 500–599 — к ошибкам сервера; 400 Bad Request используется, когда сервер отклоняет запрос из-за проблемы на стороне клиента.

После проверки удалите тестовый маршрут.

Требования к результату

Приложение должно:

- проверять данные POST /api/products;
- проверять переданные поля PATCH /api/products/:id;
- не создавать товар с некорректными данными;
- возвращать 400 для ошибки валидации;
- возвращать список обнаруженных ошибок;
- использовать AppError;
- передавать ошибки через next(error);
- возвращать 404 для отсутствующего товара;
- возвращать 404 для неизвестного маршрута;
- иметь централизованный errorHandler;
- возвращать 500 для неожиданной серверной ошибки;
- не раскрывать клиенту технические подробности внутренней ошибки.

